LinkSAGE: Optimizing Job Matching Using Graph Neural Networks

Ping Liu
LinkedIn Corporation
Mountain View, CA, USA
piliu@linkedin.com

Haichao Wei
LinkedIn Corporation
Mountain View, CA, USA
hawei@linkedin.com

Xiaochen Hou
LinkedIn Corporation
Mountain View, CA, USA
xiahou@linkedin.com

Jianqiang Shen
LinkedIn Corporation
Mountain View, CA, USA
jershen@linkedin.com

Shihai He
LinkedIn Corporation
Mountain View, CA, USA
she1@linkedin.com

Qianqi Shen
LinkedIn Corporation
Mountain View, CA, USA
qishen@linkedin.com

Zhujun Chen
LinkedIn Corporation
Mountain View, CA, USA
zhuchen@linkedin.com

Fedor Borisyuk
LinkedIn Corporation
Mountain View, CA, USA
fborisyuk@linkedin.com

Daniel Hewlett
LinkedIn Corporation
Mountain View, CA, USA
dhewlett@linkedin.com

Liang Wu
LinkedIn Corporation
Mountain View, CA, USA
liawu@linkedin.com

Srikant Veeraraghavan
LinkedIn Corporation
Mountain View, CA, USA
sveeraraghavan@linkedin.com

Alex Tsun
LinkedIn Corporation
Mountain View, CA, USA
atsun@linkedin.com

Chengming Jiang
LinkedIn Corporation
Mountain View, CA, USA
cjiang@linkedin.com

Wenjing Zhang
LinkedIn Corporation
Mountain View, CA, USA
wzhang@linkedin.com

Abstract

We present LinkSAGE, an innovative framework that integrates Graph Neural Networks (GNNs) into large-scale personalized job matching systems, designed to address the complex dynamics of LinkedIn's extensive professional network. Our approach capitalizes on a novel job marketplace graph, the largest and most intricate of its kind in industry, with billions of nodes and edges. This graph is not merely extensive but also richly detailed, encompassing member and job nodes along with key attributes, thus creating an expansive and interwoven network. A key innovation in LinkSAGE is its training and serving methodology, which effectively combines inductive graph learning on a heterogeneous, evolving graph with an encoder-decoder GNN model. This methodology decouples the training of the GNN model from that of existing Deep Neural Network (DNN) models, eliminating the need for frequent GNN retraining while maintaining up-to-date graph signals in near real-time, allowing for the effective integration of GNN insights through transfer learning. The subsequent nearline inference system serves the GNN encoder within a real-world setting, significantly reducing online latency and obviating the need for costly real-time GNN infrastructure. Validated across multiple online A/B tests in diverse product scenarios, LinkSAGE demonstrates marked improvements in member engagement, relevance matching, and member retention, confirming its generalizability and practical impact.

KDD '25, August 3–7, 2025, Toronto, ON, Canada

ACM ISBN 979-8-4007-1245-6/25/08
https://doi.org/10.1145/3690624.3709396

CCS Concepts

• **Computing methodologies** → **Neural networks**; • **Human-centered computing** → **Social networking sites**; • **Information systems** → **Recommender systems**.

Keywords

Graph Neural Networks, GNN, Transfer Learning, Recommender Systems, Job Marketplace

ACM Reference Format:
Ping Liu, Haichao Wei, Xiaochen Hou, Jianqiang Shen, Shihai He, Qianqi Shen, Zhujun Chen, Fedor Borisyuk, Daniel Hewlett, Liang Wu, Srikant Veeraraghavan, Alex Tsun, Chengming Jiang, and Wenjing Zhang. 2025. LinkSAGE: Optimizing Job Matching Using Graph Neural Networks. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '25), August 3–7, 2025, Toronto, ON, Canada.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3690624.3709396

1 Introduction

LinkedIn, the world's largest professional networking platform with over 1 billion members, serves as a vibrant hub for career development, job recruitment, and professional growth. The platform connects individuals based on commonalities like educational backgrounds and professional experiences, fostering a network where opportunities and competencies are enhanced through these connections. On average, our job product service, including recommendation and search, facilitates over 85 million job applications on our platform each week. Job matching on LinkedIn comes with its own set of unique challenges. With vast data on job opportunities and member profiles, the platform faces various complexities:

- **Dynamic Nature of Jobs**: Jobs on LinkedIn are dynamic and short-lived. This fast-paced environment demands timely

relevance in matching jobs to potential candidates, a more intricate task than in other recommender systems.
- **Sparse Engagement**: The interaction between members and job listings is often sparse and temporal, complicating engagement analysis.
- **Multidimensional Matching**: Job matching involves diverse factors including skills, education, industry experience, and seniority. Real-world scenarios often see partial matches leading to successful job placements.
- **Cold-Start Problem**: The cold-start problem in the job marketplace refers to situations where a member or job has not yet had many engagements or activities. Despite this, we still possess some profile information about them.
- **Large-Scale Evolving Data**: The platform's vast user base, exceeding 1 billion members, along with millions of jobs and companies, contributes to its large-scale heterogeneity.

Graph Neural Networks (GNN) have emerged as a promising solution to these challenges. GNNs are adept at handling dynamic relationship in real-time, managing complex relational datasets, and representing diverse data types like skills, geographies, and industries as node types. Their message-passing capabilities effectively mitigate the cold-start problem and can provide more relevant job-matching results. While GNNs have been implemented in various large-scale recommender systems, integrating them into existing product systems, often powered by deep neural networks (DNNs), remains a challenge. This paper proposes an industrial framework for applying GNN within large-scale recommender systems that are currently driven by DNNs. This framework is not only economically feasible but also ensures a non-disruptive transition to a GNN-based model. Following this framework, we have developed the largest job marketplace graph in the industry, with billions of nodes and edges, including members, jobs, and essential attributes as shown in Figure 1. Our unique contributions include:

[Application novelty] ***GNN in Job Marketplace***: Our research introduces the application of GNN to scalable job marketplaces, a domain with limited precedent for deploying GNNs at scale in an online setting. Our thorough review found no existing studies exploring GNNs within this specific context, highlighting the innovative nature of our work and its contribution to both graph neural networks and employment marketplace analytics.

[Model novelty] ***Graph Construction and Training Methodology:*** We present innovative methods in graph construction, learning, and inference. Both member and job common attributes are treated as nodes and edges to enhance information flow and preserve entity characteristics (discussion in Section 3). Compared to the prior approach [9, 31, 41] of modeling them as node features, this approach effectively handles overlapping attributes by creating direct attribute-based connections, thereby enhancing the model's contextual understanding. Meanwhile, our framework employs a novel approach by integrating inductive graph learning on a heterogeneous, evolving graph with an encoder-decoder GNN model. This approach eliminates the need for frequent GNN model retraining (which is practically impossible due to the evolving nature of the large scale, dynamic graph) while keeping the graph always up to date in near real-time for training and serving the downstream DNN models. This approach enables the seamless integration of GNN insights into established DNN models through transfer learning, resulting in significant relevance improvements with minimal impact on online latency.

[Engineer novelty] ***GNN Serving via Nearline Inference***: The framework introduces an innovative serving mechanism for the GNN encoder, incorporating it into existing DNN models within the recommender system through a nearline inference pipeline. This strategy involves precomputing the computationally intensive GNN encoder and storing its output in an in-memory feature store, which is then utilized by the DNN models. This approach effectively eliminates the need for costly real-time GNN infrastructure, achieves significant improvements in relevance performance, and ensures that latency remains in the low tens of milliseconds. The nearline inference system is particularly vital due to the vast number of jobs posted on LinkedIn daily. Without this system, the full potential of the GNN would not be realized until the next day's inference task is complete, a challenge we address in Section 4.

[Business impact] ***Relevance Matching on Equality and Segments***: The designed heterogeneous graph enables the information propagation through edges, allowing member nodes with less training data to receive significant information from its neighboring nodes that have more robust data during the training of our GNN encoder-decoder model. Our application of GNN encoder to existing neural networks has improved relevance matching across all of member base from power members to infrequent visitors historically lacking in predictive data. Our online A/B tests demonstrate significant improvements in job matching relevance, showcasing its effectiveness in various business scenarios.

By deploying LinkSAGE to production, our work not only showcases the generalization and effectiveness of LinkSAGE in applying GNN to improve various business metrics across different use cases, such as member engagement and relevance matching, but also demonstrates its potential in promoting equality and inclusivity in job recommendations. The same method can be applied to many other recommender systems.

The rest of paper is organized as follows. We discuss the related work in Section 2. We describe our graph definition in Section 3. We explain our training settings in Section 4, illustrate how to integrate GNN into online models in Section 5, then discuss and analyze our online experimental results in Section 7. We conclude our work and discuss future work in Section 8.

2 Related Work

Graph neural networks (GNNs) have demonstrated impressive capabilities in handling structured and relational datasets across various domains, including social networks, biology, chemistry, recommender systems, and computer vision [8, 12, 33, 39, 41]. Graph Convolutional Networks (GCNs) [6, 17] stand out for learning shared filter parameters from graph data. Concurrently, the GraphSAGE framework [15] offers flexible inductive aggregation options to manage dynamic neighbors in graphs. Furthermore, Graph Attention Networks (GATs) [36] utilize masked self-attentional weights across various neighbors, indicating a trend towards more complex graph structures [10, 40, 42].

In the industry, GNN applications are predominantly seen in recommender systems [1, 22, 23, 35]. Pinterest [41] pioneered the

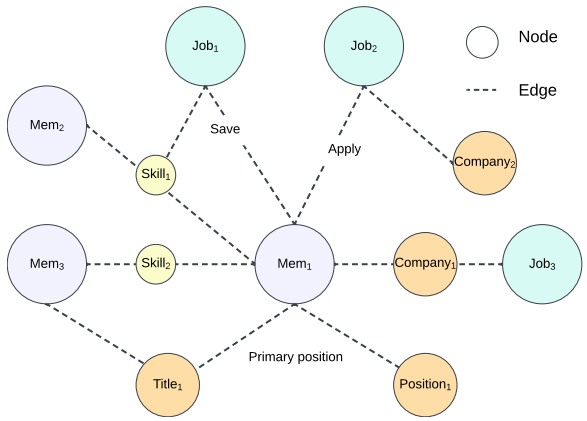

Figure 1: An example of LinkedIn's job marketplace graph. Members (also referred to as job seekers) are connected to their attributes, including Skill, Title, Company, and Position. The same connection is applied for each job. A member is connected to a job as well if there are engagements happening (e.g., save, apply, or click).

use of large-scale web-based graph neural networks for recommendations. Twitter [9, 31] developed and open-sourced the TwHIN framework, which employs pre-trained entity representations to enhance downstream recommendations, achieving notable offline and online results. Uber [2] also utilized pre-trained GNN representations in candidate selection and personal ranking models. Google Maps has recently implemented large-scale spatio-temporal interactions in its ETA predictions using these techniques [7].

The job recommender system typically mirrors the methodologies of general recommender systems [5, 34]. Content-based [16, 26] and collaborative filtering [20, 29] approaches are popular in academic research. Knowledge-based systems [14], which focus on matching and similarity in an ontology space [5, 25, 30], are unique to job recommendation. Recently, hybrid systems and deep neural networks have gained traction [27]. Qin et al. [28] leveraged rich text data from job requirements and seekers' experiences, using Recurrent Neural Networks for person-job fit prediction. [18] introduced an interpretable job-seeker fit system, providing reasoning behind recommendations.

The pursuit of equity in recommender systems has gained significant attention [38], with a growing body of research dedicated to ensuring equitable outcomes for all users [13, 19]. The issue of data sample bias in machine learning systems, stemming from various sources such as data or algorithmic models, presents a notable challenge [3]. Ensuring balanced representation and fair treatment across diverse user segments is a critical, ongoing area of focus [21]. In Section 7, we delve into the results of our A/B testing, specifically examining the system's performance in terms of equity and its impact on different user segments.

3 Graph Construction

In this section, we discuss how we construct the job marketplace graph including node types and edge types. Following a neighborhood aggregation scheme, the job marketplace graph is designed to recursively compute the representation vector of a node by aggregating and transforming representation vectors of its neighboring nodes. The performance of our GNN model is tied to the efficient flow of information between entities and hence highly depends on the underlying graph's structure. A high-quality, heterogeneous graph that encodes rich information can significantly enhance the performance of our models across the ranking system stack. We leverage our prior knowledge on hiring efficiency to design heuristic rules that guide the graph construction.

In terms of node types, we introduce 6 categories of entities to the graph: member, job, skill, title, company, and position. Nodes for skills, titles, and companies are extracted attributes from members/jobs, while the position node represents a ⟨company, title⟩ tuple which provides a clearer definition of a job opening. Based on interviews with job seekers, recruiters, and thorough data analysis on hiring outcomes, we consider these factors pivotal in determining the quality of job matching. The detailed statistics on those nodes are shown in Table 1.

Introduced edges can be categorized into 2 groups. The first group links members and jobs to their static attributes, with each edge type corresponding to a different attribute node. For instance, if a seeker is a "software engineer", we will link this seeker to title node "*software engineer*"; if a job is from company "dream inc", we will link this job to company node "*dream inc*". The second group relies on implicit connections between members and jobs, inferred from their interactions. We believe that if a member and job pair has interactions, it implies the existence of commonalities linking them together. In our graph, edges from member to job nodes symbolize user engagement (e.g., saving, applying, clicking a job), while those from job to member nodes reflect recruiter interactions (e.g., reach out to seekers). To further augment graph connectivity, we introduce reciprocal edges manually.

Our training data set contains 1B members and 50M jobs. The initial version of our graph was very sparse and there were barely any connections between member⇔job, member⇔member, or job⇔job pairs. We tackled this issue by densifying the graph through the inclusion of skill connections. In recent years, LinkedIn has embarked on a journey to help our members and customers hire for the future, making skill-first investments to foster a more equitable and efficient job market. However, it's important to note that, on average, a member possesses around 17-18 skills, while a job typically lists 30+ skills as requirement, many of which may not be crucial to job matching. For instance, *John* may have skills such as *HTTP*, *Java*, and *Machine Learning* in his profile, but he is likely primarily interested in jobs related to *Machine Learning*.

We identify top skills by building a scoring model[1] to assess the relevance of a particular skill to a seeker or a job. We then add links to connect these top skills to the corresponding seekers or jobs. Skills play a critical role by facilitating the exchange of information between members with similar skill sets and enabling information flows from jobs to members (or vice versa) when a member acquires skills relevant to a particular job. We have found that GNN, when properly designed to incorporate skill-level information, can foster meaningful connections between skills, members, and jobs, leading

[1] https://www.linkedin.com/blog/engineering/data/building-maintaining-the-skills-taxonomy-that-powers-linkedins-skills-graph

Node Type	# of Nodes	Node Type	# of Nodes
member	1B	job	50M
title	25K	position	195M
company	25M	skill	41K

Table 1: Statistics of Nodes in LinkedIn job marketplace graph

Edge Type	# of Edge	Edge Type	# of Edge
member-title	1B	job-title	46M
member-company	966M	job-company	42M
member-position	139M	job-position	41M
member-skill	1.2B	job-skill	33M
recruiter interaction	26M	seeker engagement	2.7B

Table 2: Statistics of Edges in LinkedIn job marketplace graph

to enhanced model performance. Table. 2 shows that on average a member is linked to 1.2 top skill, and job is linked to 0.67 top skill in our graph. The resulting graph is vast, encompassing billions of nodes and edges, with detailed statistics available in Table 2 [2].

We believe that the effectiveness of GNN can be enhanced by leveraging skill connections to improve graph construction capabilities and by inferring skill signals. Our offline GNN model evaluations, which included skill nodes and focused on extracting top skills relevant to both jobs and members, demonstrated a notable improvement. Specifically, we observed a +1.5% increase in recall in our offline models compared to the baseline models that lacked skill nodes in the job marketplace graph.

4 Model Training

In this section, we discuss how we prepare the graph data and the model architecture. We also explain the design of our model training process in production.

4.1 Data Preparation

To train the GNN model, we need to construct both graph data and label data. The graph data construction is highly important and extensively discussed in Section 3. Once the graph is formed, the next step is to employ a graph engine to store the graph data and provide real-time graph sampling information during GNN model training. Several products cater to this purpose, such as DeepGNN [32], PyG [11], and DGL [37]. In our case, we chose DeepGNN for its scalability, compatibility, and support for various graph sampling algorithms including multi-hop random sampling, weighted sampling, and Personalized PageRank (PPR) sampling [4]. The graph data is stored in binary format in DeepGNN, so we need to utilize the data converter component provided by DeepGNN to transform the raw graph data into binary format.

The label data preparation is closely related to how the job recommendation task is modeled. It's natural to formulate the job recommendation application as a link prediction problem, wherein the objective is to predict the existence of an edge between a member node and a job node. Consequently, the label data is structured as tuples in the form of (memberId, jobId, label), where the label field indicates whether to recommend the job to the member.

[2]The anonymized, sampled graph is available via https://github.com/pliu19/LinkSage-KDD

4.2 Model Architecture

As illustrated by Figure 2, our Graph Neural Network model adopts an encoder-decoder architecture. The encoder, specifically, utilizes the GraphSAGE algorithm [15] for its ability to handle large-scale, evolving graphs inductively. Our current encoder supports both mean aggregator (GCN) and attention aggregator (GAT).

Let $\mathcal{M}$ be the matrix of member embeddings in a batch, $\mathcal{J}$ be the matrix of job embeddings in a batch, and score(i, j) be the score for the i-th member and the j-th job.

Mean aggregation updates embeddings as

$$\mathcal{M}_i = \frac{1}{|\mathcal{N}(i)|} \sum_{n \in \mathcal{N}(i)} f(\text{features}(n))$$

$$\mathcal{J}_j = \frac{1}{|\mathcal{N}(j)|} \sum_{n \in \mathcal{N}(j)} f(\text{features}(n)),$$

and attention-based aggregation updates embeddings as

$$\mathcal{M}_i = \sum_{n \in \mathcal{N}(i)} \alpha(i, n) \times f(\text{features}(n))$$

$$\mathcal{J}_j = \sum_{n \in \mathcal{N}(j)} \alpha(i, n) \times f(\text{features}(n)),$$

where $\mathcal{N}(i)$ denotes the set of neighbors of node i, $f(\text{features}(n))$ represents the feature transformation function, $\alpha(i, n)$ is the attention score between node i and its neighbor n, computed as part of the attention mechanism.

In this manner, the encoder aggregates features from neighbors to produce the embedding matrices $\mathcal{M}_i$ and $\mathcal{J}_j$ for members and jobs, respectively. Following the encoder, these embeddings are fed into the decoder to calculate prediction scores. Our model supports various decoders: the Multilayer Perceptron (MLP) decoder, the cosine decoder, and the in-batch negative sampling decoder.

Let's illustrate with the example of in-batch negative sampling. The prediction score between a member and job pair in the batch is calculated using the dot product of their embeddings, expressed as:

$$\text{score}(i, j) = \mathcal{M}_i \cdot \mathcal{J}_j$$

where $\mathcal{M}_i$ is the embedding for the i-th member and $\mathcal{J}_j$ is the embedding for the j-th job.

The loss function for this model is the cross-entropy loss, which compares the predicted scores against the true labels. Given the member set S_M and the job set S_J, the total loss over the training data is expressed as:

$$\text{Loss} = -\sum_{i \in S_M} \sum_{j \in S_J} (y_{ij} \log(\sigma(\text{score}(i, j))) + (1 - y_{ij}) \log(1 - \sigma(\text{score}(i, j)))),$$

where σ is the sigmoid function and y_{ij} is the true label for the pair (i, j), which is 1 if the pair is a positive example and 0 otherwise.

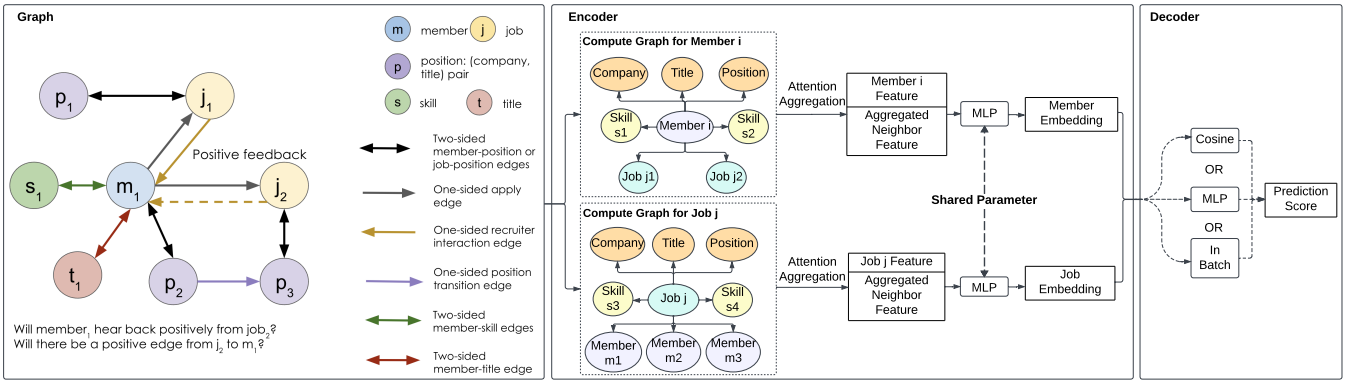

Figure 2: Model Architecture. On the left is an illustration of the graph structure and its node types, while on the right is the corresponding encoder-decoder architecture.

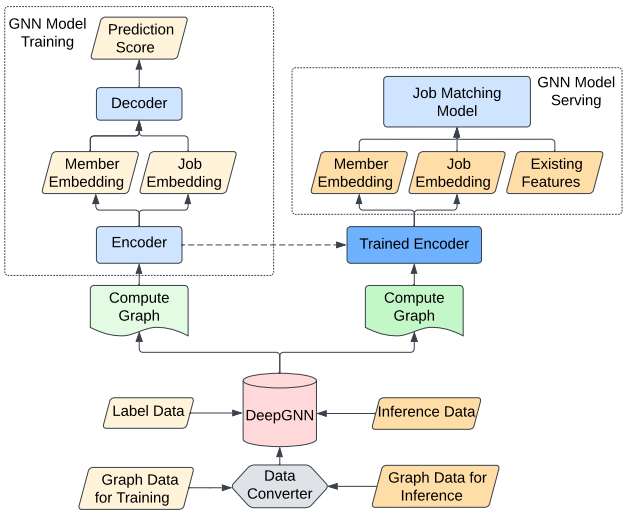

Figure 3: Model training and serving. The DeepGNN serves as the graph engine to store data and sample neighbors. The left branch illustrates the GNN training process, while the right branch uses the trained encoder to serve the GNN embeddings to job matching models.

4.3 Training and Optimization

The overall design of the model training pipeline is illustrated in the left section of Figure 3. The process begins with input label data, structured as tuples of ⟨memberId, jobId, label⟩. These tuples are then processed where memberId and jobId are utilized as query nodes in DeepGNN. DeepGNN, leveraging the existing graph data, constructs a compute graph centered around these query nodes. This compute graph comprises the query node (either a member or a job), neighbors sampled according to a predefined sampling algorithm, and their associated features.

During the iteration of training the model to get better performance, a key insight emerged: enhancing graph connectivity is crucial. This led to a focused effort on refining the design of the graph's edges. Our observations indicated that optimal performance was achieved when certain edge configurations were employed. Specifically, we found that creating bidirectional edges between members and their titles as well as their skills was most effective. Additionally, we maintained bidirectional edges for position transitions. Furthermore, we established bidirectional edges between both members and jobs to their respective positions. This structured approach in edge design as illustrated by the graph section of Figure 2 significantly contributed to the enhanced performance of our model by fostering a more interconnected and robust graph structure. Due to the page limit, please find the details of training hardware, framework, parameter selection, as well as overhead analysis in the Appendix Section.

5 Model Serving

The GNN encoder, once trained, is integrated into the ranking models for serving online traffic, as illustrated in the right section of Figure 3. To improve personalization experience to our members, we built a nearline infrastructure to continuously update GNN embeddings. GNN serves as a platform for our AI engineers, facilitating feature engineering and seamless integration of signals into our ecosystem.

5.1 Integration with Ranking Models

To meet stringent latency requirements in our online environment, the actual ranking model that is served online is a light DNN-based model by plugging in the well trained GNN encoder. The right part of Figure 3 illustrates the process. A job matching model concatenates the member and job embedding from the GNN encoder model with other relevant features and is trained with its respective objective function. To avoid label leakage, we ensure that the GNN model's training and graph data predate those used for training the job matching model.

This approach eliminates the need for real-time computation of member and job embeddings. Instead, they can be precomputed

offline and cached in a key-value store. In the offline inference flow, the most recent activities of both members and jobs are integrated into the graph data and used to update their embeddings. During the online inference, we retrieve them based on their respective IDs as input features to the ranking model. This significantly reduces online latency, from seconds to just tens of milliseconds. The implementation detail is described in Section 5.2.

The adaptability of this design extends to incorporating the trained GNN encoder into various downstream models through transfer learning. This approach not only showcases the flexibility of our model but also its capacity for generalization, leading to significant improvements in a range of online metrics across multiple applications.

5.2 Nearline Inference Framework

"*Nearline*" model inference, distinct from "*real-time*", offers a more lenient time frame for event handling. Real-time model inference involves immediate computations, typically within milliseconds, while in the nearline system, there is typically an acceptable delay of a few seconds or even a few minutes. Our previous "offline" approach involved daily batch computations to generate member and job embeddings and store them in the key-value store, leading to a 24-hour delay for newly created jobs to receive their GNN embeddings. This delay also affected the reflection of member engagement activities in the job marketplace graph. LinkedIn has a highly dynamic repository of job postings, each with an average lifespan of ~3 weeks. Job seekers are active on our platform and over 50% of them resume their job-seeking sessions within 10 minutes at least once per month. To enhance the experience for our members, we require more timely updates for embeddings, transitioning from offline to nearline processing.

Implementing a nearline GNN inference pipeline faces several challenges, particularly given that we still need the graph engine serving the query request of graph data in real-time. Key difficulties include: (a) The job marketplace graph size ranges from 3-5TB, requiring equivalent physical memory for loading, which is costly. (b) Real-time updates are operationally demanding due to their frequency. (c) The graph engine's dependence on significant resources introduces I/O bottlenecks, resulting in increased latency – a critical concern in industrial production.

To tackle these challenges, we introduced a nearline inference framework that utilizes sequential joining and an in-memory NoSQL database-based feature store. This approach creates a "stateful" job marketplace graph for production without the need for a real-time, fully operational graph engine. This streamlined solution is feasible and practical because, during inference, only the neighbors and their input features are required, rather than the full spectrum of capabilities like temporal processing and sampling typically associated with a comprehensive graph engine.

Taking job embedding as an example, Figure 4 illustrates the workflow for job embedding in our nearline inference system. The system is triggered in two scenarios: (1) when a recruiter creates a new job posting, and (2) when new neighbors (e.g., members who applied/saved/clicked on a job posting) are introduced to an existing job. These triggers are monitored via Kafka messagings. Neighbor data is continuously updated and stored in NoSQL-type feature stores keyed by job IDs. Our configuration includes multiple feature stores that monitor job neighbors per node type, accompanied by a secondary store for the input features of each neighbor. By defining a sequential join in the NoSQL store, we fetch nodes, neighbors, and input features, perform necessary transformations, and feed them as features into the nearline GNN model inference flow. The resulting embeddings are pushed to an online feature store used as precomputed features for Job Matching Models (including job ranking and candidate selection models).

This nearline inference system, currently operational in our production environment, notably reduces latency to tens of milliseconds, with a peak QPS (Query per Second) exceeding 5K. It serves as our foundation for feature engineering effort and we observed notable business improvement with the implementation of this system.

6 GNN as a Platform for Feature Engineering

Our vision for GNN training and deployment aims to enhance our matching capabilities into a cohesive "job marketplace stack" at LinkedIn.

We maintain various ranking models to connect members with opportunities like jobs, recruiters, and courses. We find that certain signals – particularly those reflecting a member's experience and expertise – are universally beneficial across these models. Traditionally, the incorporation of new signals into our system has been gradual, often beginning with a single model before extending to others. To improve AI productivity, we implement GNN as a new horizontal-first working model where multiple signals can be incorporated rapidly across the full suite of AI models for our job marketplace.

We have leveraged GNN infrastructure to seamlessly introduce new signals into the marketplace's ranking models, marking a pivotal shift in our development strategy. This shift prioritizes GNN for the initial integration of signals, with AI engineers across various teams contributing. By employing a development model akin to Git branching, engineers have the flexibility to independently explore and assess the impact of new signals on their specific models by branching out GNN model. These efforts are then merged into a main GNN model branch, which, once put into production, necessitates the retraining of downstream models to harness these advancements fully.

The GNN framework proved highly effective in our signal integration workflow. By integrating signals into GNN as a primary step, engineers from various teams successfully achieved significant improvement in a wide array of applications. This cross-functional collaboration facilitated through GNN not only streamlined the integration process but also amplified the overall impact on our product suite. For an in-depth analysis of these advancements, please refer to Section 7.

7 Experiments

We evaluated the generalization and effectiveness of our GNN system on various LinkedIn jobs marketplace downstream models through online A/B tests. The experiments focus on machine learning models for candidate selection and personalized ranking. Significant results are reported in comparison to the baseline, determined

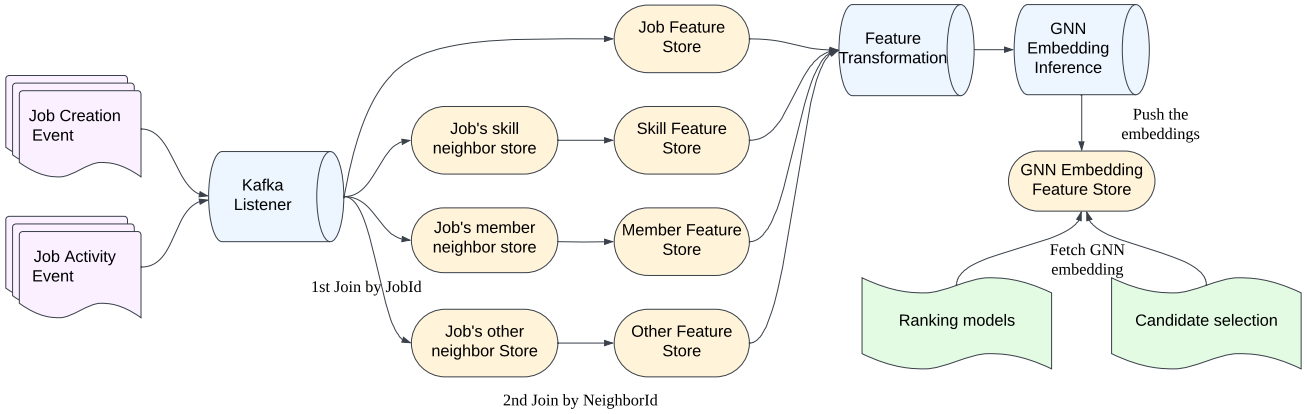

Figure 4: The working flow of nearline inference pipeline for job embeddings.

by a hypothesis t-test with a p-value less than 0.05 in two-tailed testing. Major metric definitions can be found in Table 3.

7.1 Top Applicant Jobs (TAJ)

The Top Applicant Job (TAJ) model is designed for LinkedIn Premium members. It aims to increase recruiter interactions (emails, messages, phone calls) following job applications. We approach this by treating recruiter interaction prediction as a link prediction problem, utilizing inferred GNN encoded member representation for personalization in the TAJ ranking model.

Table 4 presents the outcomes of incorporating GNN encoded member representation into the TAJ model. Data over a 12-week period reveals a notable increase in engagement among Premium members. We observed a rise in recruiter interaction metrics such as Positive Hearing Back rate, aligning with our optimization goals. Premium hence gained more value through their membership and were more willing to renew their membership.

In our second A/B test conducted a few months later, we updated the GNN model by enhancing the graph with more recruiter-member edges. Again, this led to a significant rise in positive interaction between Premium job seekers and recruiters. A +2.2% relative increase in the survival rate (transition from free to paid subscription) was likely related to such additional value offered by our platform.

7.2 Job You May Be Interested In (JYMBII)

Jobs You May Be Interested In (JYMBII) is LinkedIn's primary personalized service to recommend jobs to member. We conducted a comprehensive A/B test to measure the job seeker engagement from top of funnel job seeking sessions to bottom of the funnel focusing on the volume of qualified applications.

Table 6 displays the key improvements in the JYMBII model after incorporating GNN encoder. Notable enhancements include a +2.2% relative increase in Qualified Applications (QA) and a +0.3% relative rise in QA rate. The growth in job session metrics reflects better relevance of top recommended jobs, while a decreased job Dismiss to Apply ratio indicates higher member satisfaction. This experiment demonstrates the substantial ecosystem-level impact of skill-based GNN model in the job marketplace.

Table 7 shows the A/B test result improvement of segments of members lacking in predictive data after incorporating GNN encoder. In this experiment, we study the model performance on Opportunistic Job Seeker (casually looking) and Urgent Job Seeker (at least 1 apply in past 4 weeks). Notable enhancements include a +3.2% relative increase and a +2.6% relative rise in Qualified Applications (QA) respectively. This improvement demonstrates the GNN model improves the model performance across all the segments. Note that online metric lift in those segments historically lacking in predictive data is even higher, which is hard to move by any other techniques we have applied so far in LinkedIn's Job Recommender System.

7.3 Job Search

Table 8 presents the key metrics from our experiment incorporating GNN encoder to the Job search ranking product, another key vehicle for job marketplace. Compared to the aforementioned JYMBII experiment, this online A/B test demonstrated a greater improvement in job seeker relevancy, evidenced by successful job search sessions and the Apply to Job View ratio. While there was no increase in Qualified Applications, we observed a rise in overall application volume and positive recruiter interactions.

7.4 Embedding-based Retrieval (EBR)

Table 9 presents the key metrics from our experiment where we incorporated the GNN encoder into Embedding-based Retrieval (EBR) for the candidate generation of Job Search. This integration enhanced the relevancy and precision of job recommendations in both the Organic and Promoted channels via online A/B test.

7.5 Ablation Study of Nearline Inference

Table 10 presents a ablation study of adding GNN encoder of job in nearline vs offline in the DNN based L2 ranking model. We observed

Metric Name	Definition
Job Sessions	Number of jobs touching sessions as defined as jobs ecosystem in LinkedIn.
Qualified Applications (QA)	Applications to jobs that have owners where the applicant was targeted or matched to jobs' attributes.
Qualified Applications Rate	The ratio between number of QA and number of total applies
Dismiss To Apply Ratio	The ratio between number of total dimiss actions and number to total apply clicks

Table 3: Online Metrics Definition

Metric Name	Impact
Company Follows	+1.8%
Positive Hearing Back rate	+1.0%
Onsite Application Positive Rating Rate	+1.3%
Renewal Rate	+0.3%

Table 4: Relative metrics of the first online A/B test in TAJ.

Metric Name	Impact
Survival Rate	+2.2%
Apply Clicks Positive Interaction	+20.0%

Table 5: Relative metrics of the second online A/B test in TAJ.

Metric Name	Impact
Qualified Applications	+2.2%
Qualified Applications Rate	+0.3%
Job Sessions	+0.6%
Dismiss To Apply Ratio	−6.0%

Table 6: Relative metrics of the online A/B test in JYMBII.

Metric Name	Impact
Qualified Applications - Opportunistic	+3.2%
Qualified Applications - Open to Job	+2.8%
Qualified Applications - Urgent	+2.6%
Dismiss To Apply Ratio - Opportunistic	−13.8%
Dismiss To Apply Ratio - Open to Job	−24.2%
Dismiss To Apply Ratio - Urgent	−25.3%

Table 7: Relative metrics of the online A/B test for different segments of members.

Metric Name	Impact
Total Applies	+0.5%
Apply to view ratio	+0.5%
Successful job search sessions	+0.6%
Onsite apply positive interactions	+0.8%

Table 8: Relative metrics of the online A/B test in Job Search.

Metric Name	Impact (Organic)
Successful Job Search Sessions	+2.4%
Apply Clicks	+1.5%
Apply To Viewport Ratio	+0.9%
CTR	+1.5%
Metric Name	Impact (Promoted)
Successful Job Search Session	+1.1%
Apply Click	+0.4%
Apply To Viewport Ratio	+0.9%
CTR	+1.8%

Table 9: Relative metrics of the online A/B test in EBR.

Metric Name	Impact
Job Sessions	+0.1%
Apply Clicks	+0.6%
Successful Job Search Session	+0.8%

Table 10: Relative metrics of the online ablation A/B test of Nearline inference.

lift in both JYMBII and Job Search. With the models being able to capture signals nearline, members tend to be more engaging with the relevance content and spend more time on the product.

7.6 Summary of Learning from Experiments

Our GNN experiments in key job products, like JYMBII, showed that one model enhancement can lead to significant online impacts in A/B tests across various models. Specifically, A/B testing on job ranking models revealed a notable improvement in hiring outcomes – a remarkable feat. Additionally, search relevance saw considerable gains. These advancements benefited a wide range of members, from frequent users to casual ones, significantly improving the engagement of the latter who are often challenged by the cold start problem in machine learning.

Moreover, equity improvements via the Job Match feature were not just maintained but boosted through GNN experiments, leading to better hiring results and efficiency for groups with limited predictive data. GNN effectively leveraged its network structure to distribute historical information, significantly aiding these groups.

8 Conclusion and Future Work

We have presented a novel framework, LinkSAGE that integrates Graph Neural Networks (GNN) into LinkedIn's large-scale job

matching systems, overcoming unique challenges associated with one of the largest professional networks. Our approach utilizes a vast job marketplace graph, the first of its scale in the industry, featuring billions of nodes and edges. This graph is not only expansive but also rich in diversity, encompassing various job-related attributes that form a complex, interconnected network. A key aspect of LinkSAGE is its training and serving methodology, which effectively combines inductive graph learning with an encoder-decoder GNN model mining a heterogeneous, near real-time evolving graph. This system integrates GNN insights into existing deep neural network models through transfer learning, significantly enhancing performance while minimizing online latency. This eliminates the need for costly real-time GNN infrastructure, marking a first in the realm of large-scale recommender systems.

LinkSAGE has shown remarkable improvements in relevance matching and has been particularly beneficial for diverse user groups, including active job seekers and infrequent visitors. Importantly, it has significantly improved hiring outcomes and efficiency for groups with limited predictive data. We have conducted various experiments on LinkedIn job marketplace, where multiple practical methods are used for graph definition, online deployment, latency optimization, etc. The deployment of this framework across various LinkedIn products has been validated through multiple online A/B tests, demonstrating its effectiveness in boosting user engagement, relevance matching, and member retention. Our work represents a significant stride in the application of GNNs in industrial-scale recommender systems. By addressing key challenges and leveraging the unique properties of GNNs, we have opened new avenues for enhancing job matching services.

Looking ahead, our focus will be on further enhancing the scalability and efficiency of LinkSAGE. We aim to explore more advanced graph learning algorithms and optimization techniques that can handle even larger datasets and more complex network structures. Another area of interest is the continual improvement of the equity aspect of our model, ensuring that the benefits of GNNs are extended to an even broader range of user segments. Additionally, we plan to investigate the potential of integrating other emerging machine learning technologies with our GNN framework to further refine our job recommendation and many other recommender systems in LinkedIn.

References

[1] Fedor Borisyuk, Shihai He, Yunbo Ouyang, Morteza Ramezani, Peng Du, Xiaochen Hou, Chengming Jiang, Nitin Pasumarthy, Priya Bannur, Birjodh Tiwana, et al. 2024. Lignn: Graph neural networks at linkedin. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4793–4803.

[2] Avishek Joey Bose, Ankit Jain, Piero Molino, and William L Hamilton. 2019. Meta-graph: Few shot link prediction via meta learning. *arXiv preprint arXiv:1912.09867* (2019).

[3] Jiawei Chen, Hande Dong, Xiang Wang, Fuli Feng, Meng Wang, and Xiangnan He. 2023. Bias and debias in recommender system: A survey and future directions. *ACM Transactions on Information Systems* 41, 3 (2023), 1–39.

[4] Eli Chien, Jianhao Peng, Pan Li, and Olgica Milenkovic. 2021. Adaptive Universal Generalized PageRank Graph Neural Network. In *International Conference on Learning Representations*. https://openreview.net/forum?id=n6jl7fLxrP

[5] Corné De Ruijt and Sandjai Bhulai. 2021. Job recommender systems: A review. *arXiv preprint arXiv:2111.13576* (2021).

[6] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. 2016. Convolutional neural networks on graphs with fast localized spectral filtering. *Advances in neural information processing systems* 29 (2016).

[7] Austin Derrow-Pinion, Jennifer She, David Wong, Oliver Lange, Todd Hester, Luis Perez, Marc Nunkesser, Seongjae Lee, Xueying Guo, Brett Wiltshire, et al. 2021. Eta prediction with graph neural networks in google maps. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. 3767–3776.

[8] Kien Do, Truyen Tran, and Svetha Venkatesh. 2019. Graph transformation policy network for chemical reaction prediction. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 750–760.

[9] Ahmed El-Kishky, Thomas Markovich, Serim Park, Chetan Verma, Baekjin Kim, Ramy Eskander, Yury Malkov, Frank Portman, Sofia Samaniego, Ying Xiao, et al. 2022. Twhin: Embedding the twitter heterogeneous information network for personalized recommendation. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 2842–2850.

[10] Yifan Feng, Haoxuan You, Zizhao Zhang, Rongrong Ji, and Yue Gao. 2019. Hypergraph neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 3558–3565.

[11] Matthias Fey and Jan E. Lenssen. 2019. Fast Graph Representation Learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.

[12] Alex Fout, Jonathon Byrd, Basir Shariat, and Asa Ben-Hur. 2017. Protein interface prediction using graph convolutional networks. *Advances in neural information processing systems* 30 (2017).

[13] Yingqiang Ge, Shuchang Liu, Ruoyuan Gao, Yikun Xian, Yunqi Li, Xiangyu Zhao, Changhua Pei, Fei Sun, Junfeng Ge, Wenwu Ou, et al. 2021. Towards long-term fairness in recommendation. In *Proceedings of the 14th ACM international conference on web search and data mining*. 445–453.

[14] Francisco Gutiérrez, Sven Charleer, Robin De Croon, Nyi Nyi Htun, Gerd Goetschalckx, and Katrien Verbert. 2019. Explaining and exploring job recommendations: a user-driven approach for interacting with knowledge-based job recommender systems. In *Proceedings of the 13th ACM Conference on Recommender Systems*. 60–68.

[15] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).

[16] Rémy Kessler, Nicolas Béchet, Mathieu Roche, Juan-Manuel Torres-Moreno, and Marc El-Bèze. 2012. A hybrid approach to managing job offers and candidates. *Information processing & management* 48, 6 (2012), 1124–1135.

[17] Thomas N Kipf and Max Welling. 2016. Semi-Supervised Classification with Graph Convolutional Networks. *arXiv preprint arXiv:1609.02907* (2016).

[18] Ran Le, Wenpeng Hu, Yang Song, Tao Zhang, Dongyan Zhao, and Rui Yan. 2019. Towards effective and interpretable person-job fitting. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*. 1883–1892.

[19] Min Kyung Lee, Anuraag Jain, Hea Jin Cha, Shashank Ojha, and Daniel Kusbit. 2019. Procedural justice in algorithmic fairness: Leveraging transparency and outcome control for fair algorithmic mediation. *Proceedings of the ACM on Human-Computer Interaction* 3, CSCW (2019), 1–26.

[20] Yujin Lee, Yeon-Chang Lee, Jiwon Hong, and Sang-Wook Kim. 2017. Exploiting job transition patterns for effective job recommendation. In *2017 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. IEEE, 2414–2419.

[21] Weiwen Liu, Jun Guo, Nasim Sonboli, Robin Burke, and Shengyu Zhang. 2019. Personalized fairness-aware re-ranking for microlending. In *Proceedings of the 13th ACM Conference on Recommender Systems*. 467–471.

[22] Zemin Liu, Trung-Kien Nguyen, and Yuan Fang. 2021. Tail-gnn: Tail-node graph neural networks. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 1109–1119.

[23] Zemin Liu, Wentao Zhang, Yuan Fang, Xinming Zhang, and Steven CH Hoi. 2020. Towards locality-aware meta-learning of tail node embeddings on networks. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. 975–984.

[24] Andrew L Maas, Awni Y Hannun, Andrew Y Ng, et al. 2013. Rectifier nonlinearities improve neural network acoustic models. In *ICML*, Vol. 30. 3.

[25] Jorge Martinez-Gil, Alejandra Lorena Paoletti, and Mario Pichler. 2020. A novel approach for learning how to automatically match job offers and candidate profiles. *Information Systems Frontiers* 22 (2020), 1265–1274.

[26] Motebang Daniel Mpela and Tranos Zuva. 2020. A mobile proximity job employment recommender system. In *2020 International Conference on Artificial Intelligence, Big Data, Computing and Data Communication Systems (icABCD)*. IEEE, 1–6.

[27] Amber Nigam, Aakash Roy, Hartaran Singh, and Harsimran Waila. 2019. Job recommendation through progression of job selection. In *2019 IEEE 6th International Conference on Cloud Computing and Intelligence Systems (CCIS)*. IEEE, 212–216.

[28] Chuan Qin, Hengshu Zhu, Tong Xu, Chen Zhu, Liang Jiang, Enhong Chen, and Hui Xiong. 2018. Enhancing person-job fit for talent recruitment: An ability-aware neural network approach. In *The 41st international ACM SIGIR conference on research & development in information retrieval*. 25–34.

[29] Michael Reusens, Wilfried Lemahieu, Bart Baesens, and Luc Sels. 2017. A note on explicit versus implicit information for job recommendation. *Decision Support Systems* 98 (2017), 26–35.

[30] Alberto Rivas, Pablo Chamoso, Alfonso González-Briones, Roberto Casado-Vara, and Juan Manuel Corchado. 2019. Hybrid job offer recommender system in a social network. *Expert Systems* 36, 4 (2019), e12416.
[31] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael Bronstein. 2020. Temporal Graph Networks for Deep Learning on Dynamic Graphs. In *ICML 2020 Workshop on Graph Representation Learning*.
[32] Alex Samylkin. 2022. DeepGNN is a framework for training machine learning models on large scale graph data. https://github.com/microsoft/DeepGNN
[33] Victor Garcia Satorras and Joan Bruna Estrach. 2018. Few-shot learning with graph neural networks. In *International conference on learning representations*.
[34] Jianqiang Shen, Yuchin Juan, Ping Liu, Wen Pu, Shaobo Zhang, Qianqi Shen, Liangjie Hong, and Wenjing Zhang. 2024. Learning Links for Adaptable and Explainable Retrieval. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 4046–4050.
[35] Xiran Song, Jianxun Lian, Hong Huang, Zihan Luo, Wei Zhou, Xue Lin, Mingqi Wu, Chaozhuo Li, Xing Xie, and Hai Jin. 2023. xgcn: An extreme graph convolutional network for large-scale social link prediction. In *Proceedings of the ACM Web Conference 2023*. 349–359.
[36] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.
[37] Minjie Wang, Da Zheng, Zihao Ye, Quan Gan, Mufei Li, Xiang Song, Jinjing Zhou, Chao Ma, Lingfan Yu, Yu Gai, Tianjun Xiao, Tong He, George Karypis, Jinyang Li, and Zheng Zhang. 2019. Deep Graph Library: A Graph-Centric, Highly-Performant Package for Graph Neural Networks. *arXiv preprint arXiv:1909.01315* (2019).
[38] Yifan Wang, Weizhi Ma, Min Zhang, Yiqun Liu, and Shaoping Ma. 2023. A Survey on the Fairness of Recommender Systems. *ACM Trans. Inf. Syst.* 41, 3, Article 52 (feb 2023), 43 pages. https://doi.org/10.1145/3547333
[39] Yongji Wu, Defu Lian, Yiheng Xu, Le Wu, and Enhong Chen. 2020. Graph convolutional networks with markov random field reasoning for social spammer detection. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 1054–1061.
[40] Xiaocheng Yang, Mingyu Yan, Shirui Pan, Xiaochun Ye, and Dongrui Fan. 2023. Simple and efficient heterogeneous graph neural network. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 10816–10824.
[41] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 974–983.
[42] Chuxu Zhang, Dongjin Song, Chao Huang, Ananthram Swami, and Nitesh V Chawla. 2019. Heterogeneous graph neural network. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 793–803.

Appendix

Training Framework and Hardware As we discussed in Section 4.1, we choose to use DeepGNN as our framework. The GNN training jobs are executed in the in-house Kubernetes based cluster, which has access to a Hadoop File System (HDFS). Each job needs to deploy a Graph Engine (GE, usually on CPU nodes) and a GNN Trainer (usually on GPU nodes) in the Kubernetes cluster. During training the DeepGNN client within the GNN Trainer queries the GEs over gRPC. The resulting data is consumed by the underlying deep learning framework (Tensorflow). Typically, each training job uses 24 Nvidia v100 GPUs with training time around 48 hours. The GE takes 20 CPU nodes with 240GB memory each to load 4TB graph data.

Offline Evaluation and Parameter Selection Our framework supports vast options for parameter selections. Since we rolled out several versions of GNN embedding for different products, the parameters might be different. We take JYMBII as an example that we discussed in the Section.7.2. The model architecture is as follows. Encoder structures as a 4-layer multi-layer perceptron (MLP) that outputs a fixed 200 dimension projection as embeddings. We choose to use Leaky Relu [24] at each neuron, and apply batch normalization with L2 normalization in the final output layer. We use a [200, 200, 200] MLP decoder with softmax function to minimize the loss as a link prediction problem.

GNN Offline Training and Inference Overhead Analysis The overhead of GNN offline training and inference has three parts: (1) querying GE to fetch the sampled graph data, (2) local data processing on CPU to convert numpy data to tensors and (3) GPU computation. The data I/O part which includes (1) and (2) is the bottleneck of current GNN offline training and inference due to the large data size per batch. Compared to regular Deep Learning model, GNN model training and inference requires hundreds of times of data since it uses hundreds of neighbor nodes' data for training and inference. In our setting with 4096 batch size, 1200 dimension features per node and sampling 100 neighbors per node, each batch ends up using 2GB data. For each batch of data, it takes 5 seconds to transmit the data to training workers (1) and 10 seconds to process it on CPU (2). While the GPU computation only causes about 0.5 second since the model size is relatively small, with around 6 million parameters. Even though we apply the multi-thread parallel data fetching and processing to speed the data I/O, the CPU power still cannot be fully utilized due to launching threads and GIL in python. The training time per step is around 6 seconds.